MSDS 555 Final Project Report

SPADE Lab: Graph Databases and Distributed Systems

Farhana Mansoor, Issac Adebayo, Michella Maddox, Charnisha Azubuike

August 10, 2025

School of Applied Computational Sciences
Meharry Medical College
1005 Dr. DB TODD JR BLVD, Nashville, TN 37208

Contents

1 Introduction 4
2 Objectives 4
3 Core Features – SPADE Prototype 5
4 System Design and Architecture 5
5 Methodology 5
5.1 Database Schema 5
5.2 Development Stack 5
6 System Workflow 5
7 Results and Analysis 5
8 Advanced Features and Possible Improvements 6
9 Real World Impact 6
10 Group Activity and Contribution 6
11 Conclusion .....

Executive Summary

This project was completed as part of the SPADE Lab (Specialized Data Processing and Analysis Environment) to explore two advanced technologies used in big data analytics: Neo4j and Apache Spark. Each tool was used to solve different types of data problems. Neo4j was chosen for its ability to manage and query highly connected data using a graph-based structure. We used it to create nodes and relationships that represent football players and events, allowing us to visually model and analyze complex connections. Apache Spark was used to process and analyze large tabular data sets using PySpark. We performed operations such as filtering, aggregating, and loading data into a MySQL database. Together, these tools demonstrated how the use of the right technology for each data structure can lead to better performance and clearer insights. The project highlights the benefits of combining graph databases with distributed computing in real-world data environments.

1 Introduction

As data continues to grow in volume and complexity, traditional databases fall short in representing deep, interconnected relationships or efficiently processing large-scale datasets. This project investigates the strengths of **Neo4j**, a graph-based database, and **Apache Spark**, a distributed data processing framework, by applying each to distinct but related tasks: modeling football player data and analyzing demographic data.

2 Objectives

- Utilize Neo4j to model real-world football relationships through nodes and Cypher queries.
- Use PySpark to process, analyze, and write structured data at scale.
- Demonstrate CSV ingestion, querying, filtering, and aggregation in both environments.
- Showcase results visually using graph visualizations and console outputs.

3 Core Features – SPADE Prototype

- Graph modeling with Neo4j: player relationships, sorting, and community detection.
- Distributed processing with PySpark: data loading, transformations, and database integration.
- MySQL data persistence using PySpark's `.write.jdbc()`.
- Clear visual representation of graph-based relationships (Figures 1–4).

...

4 System Design and Architecture

- **Technologies used**: Neo4j Desktop, Apache Spark (PySpark), MySQL, Jupyter Notebook.
- **Structure**: Neo4j was used for relationship-centric data; PySpark was used for tabular/structured data processing.

5 Methodology

For this project, we used two separate tools to demonstrate different approaches to handling and analyzing data.

First, we used Neo4j, a graph database, to model complex relationships using nodes and connections. We created a structure that allowed us to represent player data and their connections to countries and events. This was done by writing Cypher queries to define each node (such as players and tournaments) and the relationships between them (like `PLAYER_OF` or `PLAYED_AT`). We also used Cypher to sort and analyze the data and to import information directly from a CSV file.

Separately, we used Apache Spark with PySpark to process large amounts of structured tabular data. We loaded the dataset into Spark using DataFrames and applied various operations such as filtering, grouping, and counting rows. One key task was calculating the number of people under a certain age and summarizing educational levels. Finally, we connected Spark to a MySQL database and wrote the cleaned data into a table called `Adult`. This workflow showed how Spark is useful for working with large datasets in a fast and scalable way.

Appendix: Key Cypher Query for CSV Load

```
LOAD CSV WITH HEADERS FROM 'file:///FIFA.csv' AS row
CREATE (:Player {
  Rank: toInteger(row.Rank),
  Name: row.Name,
  Country: row.Country,
  Goals: toInteger(row.Goals),
  Matches: toInteger(row.Matches)
});
```

6 Database Schema

6.1 Neo4j Components

- Neo4j Desktop
- Cypher Query Language
- CSV import via `LOAD CSV WITH HEADERS`
- Neo4j Browser for visualization and query execution

6.2 Apache Spark Components

- Apache Spark (PySpark)
- Jupyter Notebook for development environment
- Spark DataFrame API & Spark SQL
- MySQL (JDBC integration for writing results to relational storage)
- Python libraries: `findspark`, `pandas` (optional for display), `mysql-connector-java` (JDBC driver)

7 System Workflow

7.1 Neo4j Workflow

1. Create initial nodes and relationships using Cypher commands.
2. Import player data from `FIFA.csv` using `LOAD CSV WITH HEADERS`.
3. Link players to countries and the FIFA World Cup event.
4. Run queries to retrieve and visualize relationships in Neo4j Browser.

7.2 PySpark Workflow

1. Initialize a `SparkSession` and load `adult.csv` into a DataFrame with schema inference.
2. Apply filtering, grouping, and sorting operations.
3. Aggregate counts by specific criteria (e.g., education level, age).
4. Export the processed DataFrame to MySQL via `.write.jdbc()`.

8 Results and Analysis

Neo4j: The Neo4j implementation successfully modeled player–country and player–event relationships using Cypher queries. The graph database structure allowed quick retrieval of relationship properties such as matches and goals, and visualizations made the network connections easy to interpret.

Neo4j Q1: Creating `PLAYER_OF` relationship between Ronaldinho and Brazil

Figure 1: PLAYER_OF relationship between Ronaldinho and Brazil.

Neo4j Q2: Creating `FOOTBALLER_OF` relationship with properties for matches and goals.

Figure 2: FOOTBALLER_OF relationship showing matches and goals.

Neo4j Q3: Creating `PLAYED_AT` relationships between top players and the FIFA World Cup.

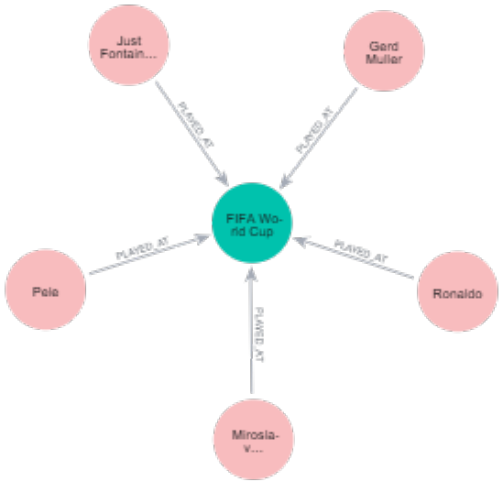


Figure 3: PLAYED_AT relationships between top players and the FIFA World Cup node.

Neo4j Q4: Loading player data from `FIFA.csv` into Neo4j as nodes with properties.

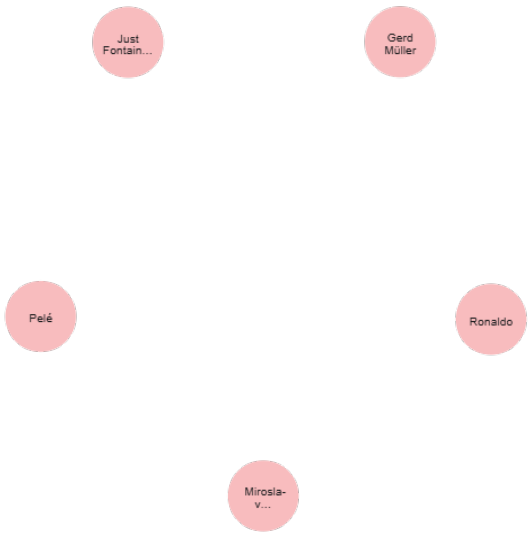


Figure 4: Result of loading players from CSV into Neo4j as nodes.

PySpark: The PySpark pipeline efficiently processed the adult.csv dataset, applying filtering, aggregation, and integration of relational databases. This demonstrated Spark's scalability and ability to work with large structured datasets.

PySpark Q1: Loading `adult.csv` into a DataFrame and displaying schema.

```
import pyspark
import findspark
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
findspark.init()
```

```
ui_port = "4045"
spark = SparkSession.builder \
    .appName("SecureSparkSession") \
    .config("spark.ui.port", ui_port) \
    .config("spark.driver.extraJavaOptions", "-Djava.security.manager=allow") \
    .getOrCreate()

df = spark.read.csv("adult.csv", header=True, inferSchema=True)
df.filter(col("Age") < 40).count()
```

```
27444
```

Figure 5: Schema of adult.csv loaded into PySpark DataFrame.

PySpark Q1b: First few rows of the `adult.csv` dataset after successful ingestion into a PySpark DataFrame.

```
+---+---------+------+------------+--------------+------------------+------------------+------------+-----+------+------------+--------
|age|workclass|fnlwgt|   education|educational-num|    marital-status|        occupation|relationship| race|gender|capital-gain|capital-l
+---+---------+------+------------+--------------+------------------+------------------+------------+-----+------+------------+--------
| 25|  Private|226802|        11th|             7|     Never-married|Machine-op-inspct|   Own-child|Black|  Male|           0|
| 38|  Private| 89814|     HS-grad|             9|Married-civ-spouse|  Farming-fishing|     Husband|White|  Male|           0|
| 28|Local-gov|336951|  Assoc-acdm|            12|Married-civ-spouse|  Protective-serv|     Husband|White|  Male|           0|
| 44|  Private|160323|Some-college|            10|Married-civ-spouse|Machine-op-inspct|     Husband|Black|  Male|        7688|
| 18|        ?|103497|Some-college|            10|     Never-married|                 ?|   Own-child|White|Female|           0|
+---+---------+------+------------+--------------+------------------+------------------+------------+-----+------+------------+--------
only showing top 5 rows

root
 |-- age: integer (nullable = true)
 |-- workclass: string (nullable = true)
 |-- fnlwgt: integer (nullable = true)
 |-- education: string (nullable = true)
 |-- educational-num: integer (nullable = true)
 |-- marital-status: string (nullable = true)
 |-- occupation: string (nullable = true)
 |-- relationship: string (nullable = true)
 |-- race: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- capital-gain: integer (nullable = true)
 |-- capital-loss: integer (nullable = true)
 |-- hours-per-week: integer (nullable = true)
 |-- native-country: string (nullable = true)
 |-- income: string (nullable = true)
```

Figure 6: First few rows of the dataset after ingestion.

PySpark Q2: Counting individuals under the age of 40.

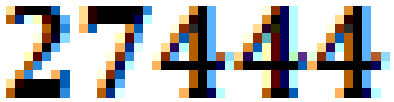
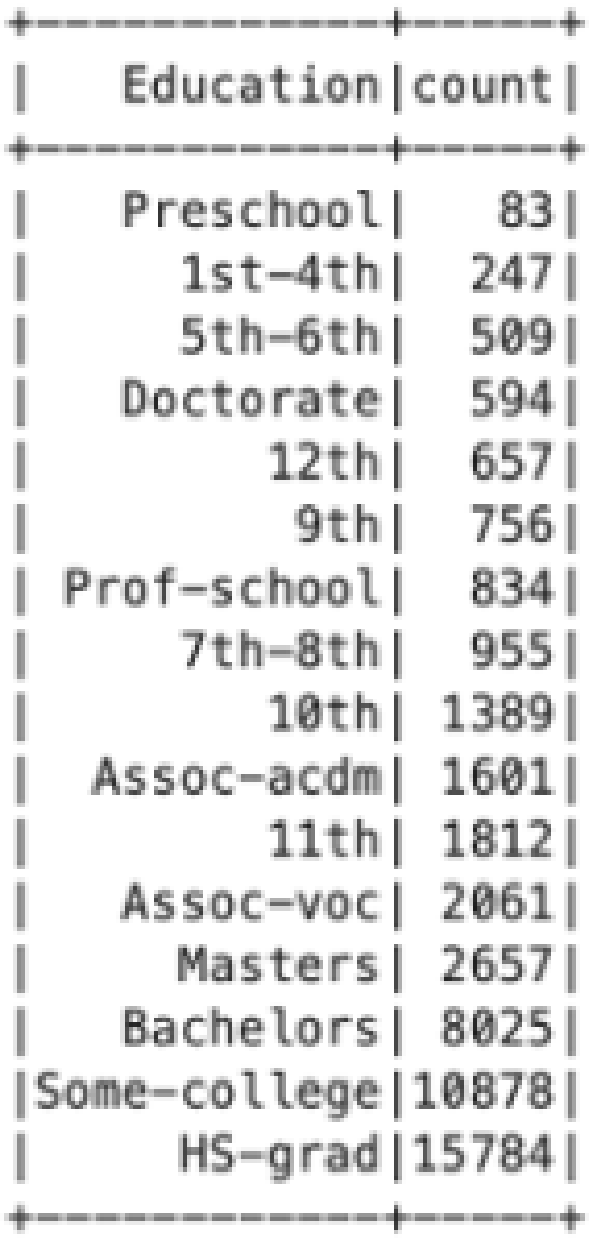


Figure 7: Count of individuals under age 40.

PySpark Q3: Aggregating counts by education level and sorting.



Education	count
Preschool	83
1st-4th	247
5th-6th	509
Doctorate	594
12th	657
9th	756
Prof-school	834
7th-8th	955
10th	1389
Assoc-acdm	1601
11th	1812
Assoc-voc	2061
Masters	2657
Bachelors	8025
Some-college	10878
HS-grad	15784

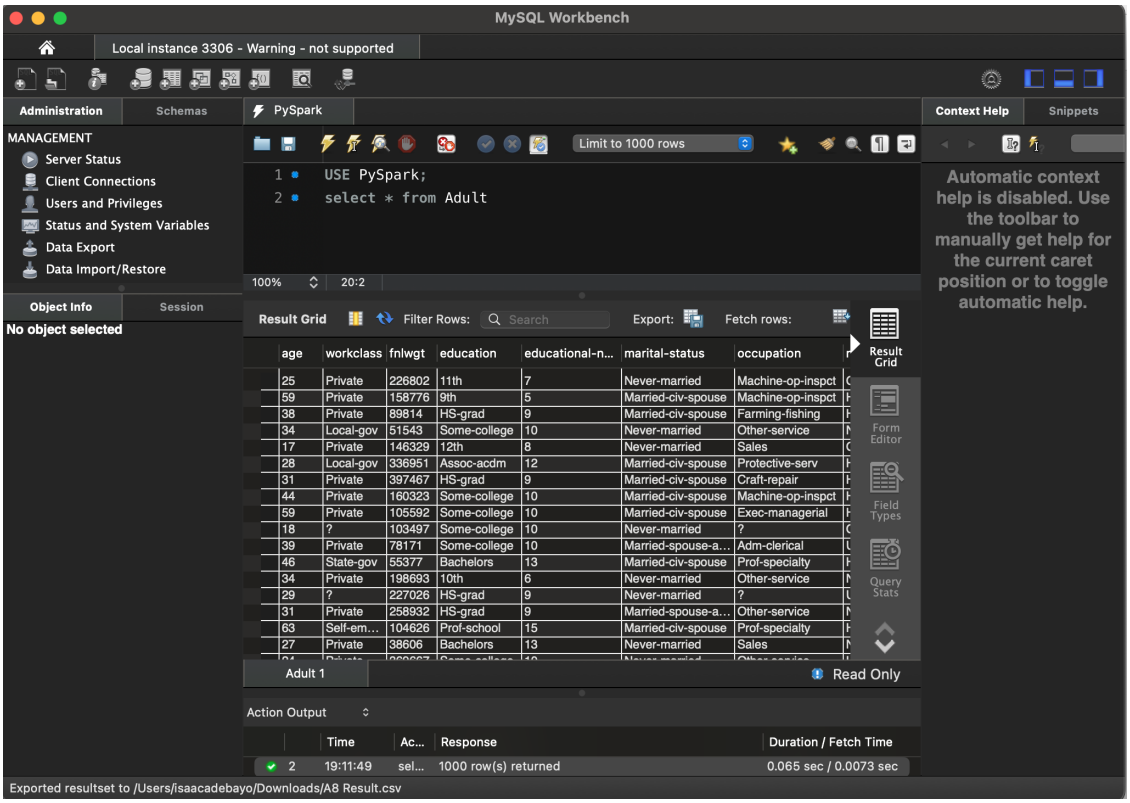
Figure 8: Counts by education level, sorted.

PySpark Q4: Exporting DataFrame to MySQL and verifying load.

```
[1]: import os
     import pyspark
     import findspark
     import mysql.connector
     from pyspark.sql import SparkSession
     from pyspark.sql.functions import col
     findspark.init()

[ ]:

[2]: connection = mysql.connector.connect(host = 'localhost',
                                          user = 'root',
                                          passwd = '!Isaac09Caasi14@-Isaac.Adebayo',
                                          db ='PySpark')

     jdbc_url = "jdbc:mysql://localhost:3306/PySpark"
     user_name = os.environ.get('DB_USER')
     password = os.environ.get('DB_PASS')

[3]: jdbc_driver_path = "/Users/isaacadebayo/Downloads/mysql-connector-j-9.2.0/mysql-connector-j-9.2.0.jar"

     mysql_url = "jdbc:mysql://localhost:3306/PySpark"
     mysql_properties = {
             "user": 'root',
             "password": '!Isaac09Caasi14@-Isaac.Adebayo',
             "driver": "com.mysql.cj.jdbc.Driver"
         }
     spark = SparkSession.builder \
         .appName("PySparkMySQL") \
         .config("spark.jars", jdbc_driver_path) \
         .config("spark.driver.extraJavaOptions", "-Djava.security.manager=allow") \
         .getOrCreate()
     df = spark.read.csv("adult.csv", header=True, inferSchema=True)
```

Figure 9: PySpark Q4 code for exporting `adult.csv` DataFrame to MySQL as the `Adult` table using `.write.jdbc()`.

PySpark Q4b: Exporting DataFrame to MySQL and verifying load.



age	workclass	fnlwgt	education	educational-n...	marital-status	occupation
25	Private	226802	11th	7	Never-married	Machine-op-inspct
59	Private	158776	9th	5	Married-civ-spouse	Machine-op-inspct
38	Private	89814	HS-grad	9	Married-civ-spouse	Farming-fishing
34	Local-gov	51543	Some-college	10	Never-married	Other-service
17	Private	146329	12th	8	Never-married	Sales
28	Local-gov	336951	Assoc-acdm	12	Married-civ-spouse	Protective-serv
31	Private	397467	HS-grad	9	Married-civ-spouse	Craft-repair
44	Private	160323	Some-college	10	Married-civ-spouse	Machine-op-inspct
59	Private	105592	Some-college	10	Married-civ-spouse	Exec-managerial
18	?	103497	Some-college	10	Never-married	?
39	Private	78171	Some-college	10	Married-spouse-a...	Adm-clerical
46	State-gov	55377	Bachelors	13	Married-civ-spouse	Prof-specialty
34	Private	198693	10th	6	Never-married	Other-service
29	?	227026	HS-grad	9	Never-married	?
31	Private	258932	HS-grad	9	Married-spouse-a...	Other-service
63	Self-em...	104626	Prof-school	15	Married-civ-spouse	Prof-specialty
27	Private	38606	Bachelors	13	Never-married	Sales

Figure 10: MySQL `Adult` table populated from PySpark export, verified with `SELECT * FROM Adult`.

9 Advanced Features and Improvements

- Add a web dashboard using Streamlit or Flask.
- Use Neo4j Bloom for advanced graph visualization.
- Apply MLlib models to the PySpark dataset.

10 Real-World Impact

- **Sports Analytics:** Player and team relationship modeling.
- **Demographic Analytics:** Age and education filtering for workforce or health policy.
- **Enterprise Readiness:** Demonstrates full-stack integration using graphs and distributed systems.

11 Group Activity and Contributions

- **Farhana Mansoor** – Responsible for Neo4j tasks (A1.txt through A4.txt)
- **Issac Adebayo** – Responsible for PySpark tasks (A5.ipynb through A8.ipynb)
- **Michella Maddox** – Led the demo and created the PowerPoint presentation
- **Charnisha Azubuike** – Wrote and formatted the technical report

12 Conclusion

By combining the strengths of Neo4j and Apache Spark, this project showcases how graph and distributed systems can effectively solve different types of big data challenges. This dual-approach model is critical in modern data pipelines where both relationships and scalability matter.